

Document Summary

This note covers the essential components of a deep learning pipeline, including data handling, preprocessing, and transformation techniques for preparing data suitable for neural networks. It explores Feed-Forward Neural Networks (FFNN) and Recurrent Neural Networks (RNN) implementations in PyTorch, with practical code examples, architecture details, and best practices for training and optimization. Key challenges like data leakage, class imbalance, and long-term dependencies are addressed to ensure robust model development.

Deep Learning Architectures: Data Pipeline, Preprocessing, FFNN, and RNN in PyTorch

Agenda

- Basic Theory
- Coding Example!
- About
 - Data Handling
 - **Deep Learning Architectures**
 - Feed-Forward Neural Network (**FFNN**)
 - Recurrent Neural Network (**RNN**)
 - Graph Neural Network (**GNN**)
 - Convolutional Neural Network (**CNN**)

Deep Learning Pipeline

The **deep learning pipeline** simplifies feature extraction but requires data knowledge and preparation before model input. To ensure smooth progression, this pipeline breaks down the process into sequential steps, starting from raw data and leading to model training.

Components of the Pipeline

- Split
- Transformation
- Data Handling

- **Pre-Processing**

The following diagram illustrates the logical flow of these components in the deep learning pipeline:



Data Handling

Data handling prepares raw data for deep learning models. This step involves organizing and structuring the data to make it compatible with subsequent transformations and model inputs, ensuring efficiency and consistency.

Data Handling

The process of organizing and structuring raw data to ensure compatibility with deep learning models and subsequent pipeline steps.

Pre-Processing

Pre-processing ensures data quality to avoid performance issues. By addressing imperfections early, this step prevents downstream errors in the pipeline.

- **Data cleaning:**
 - Noise and outliers affect the pipeline.
 - **Noise less, outlier more.**
 - For instance, random noise in sensor data can skew predictions, while outliers like extreme values in network traffic might indicate anomalies but need careful handling.

Practical Example of Data Cleaning

In sensor data, random noise might be filtered using smoothing techniques, while outliers in network traffic could be investigated for anomalies before inclusion.

- **Handle redundancy:**
 - Duplicates inflate point importance.
 - Reduces generalization.
 - Removing duplicates ensures each data point contributes equally, improving model robustness.

Important Caution on Redundancy

Failing to remove duplicates can lead to biased models that overemphasize repeated instances, harming generalization.

- **Handle missing values:**

- Missing values unusable by neural networks.
- Options:
 - Eliminate points.
 - Estimate values.
 - Ignore features.
 - Replace with possible values (weighted by probabilities).
 - If sensible.
- Example: In a dataset with missing temperature readings, imputation using the mean might be sensible for continuous data, while dropping rows could work for small gaps.

Practical Example of Handling Missing Values

For a temperature dataset, use mean imputation: a missing 25°C reading could be filled with the average of surrounding values.

- Tools: Use `Pandas` for pre-processing.

Code Example Using Pandas to Handle Missing Values

```
import pandas as pd
import numpy as np

# Sample data with missing values
data = pd.DataFrame({'A': [1, 2, np.nan, 4], 'B': [5, np.nan, 7, 8]})

# Impute missing values with mean
data_filled = data.fillna(data.mean())
print(data_filled)
```

Data Transformation

Data transformation varies by data type. This process converts raw attributes into formats suitable for neural network inputs, with techniques tailored to categorical, ordinal, or numerical data.

- **Categorical attributes that are nominal:**
 - Examples: colors, country name, encryption algorithm, internet port, words.
- **Categorical attributes that are ordinal:**
 - Examples: educational level, cloth size, access control level, threat level.
- **Numerical attributes:**
 - Examples: temperature, height, byte sent, number of packets, password attempts.

Data Transformation: Categorical and Ordinal Attributes

- **One-hot-encoding:**
 - Converts categorical value to binary vector.
 - 1 input neuron per possible value.
 - **Pros:** Simple, effective for small categorical data.
 - **Cons:** Input neurons explode for high-cardinality features.
 - Example: Text? English 171k+ words → 171k neurons?

Important Caution on One-Hot Encoding

High-cardinality features can lead to dimensionality explosion, increasing computational costs dramatically.

- **Label encoding:**
 - Assigns integer to each category.
 - 1 input neuron per feature.
 - **Pros:** Simple, memory efficient.
 - **Cons:** Implies ordinal relationships, misleading model.
 - Use for ordinal data or textual labels.

Example of Label Encoding

<i>ID</i>	<i>Access Control Level</i>	<i>Encoded Access Control Level</i>
1	Medium	2

<i>ID</i>	<i>Access Control Level</i>	<i>Encoded Access Control Level</i>
2	High	3
3	Low	1
4	Medium	2

Code Snippet for Label Encoding Using Scikit-Learn

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd

data = pd.DataFrame({'Access': ['Medium', 'High', 'Low', 'Medium']})
le = LabelEncoder()
data['Encoded'] = le.fit_transform(data['Access'])
print(data)
```

- **Embeddings:**
 - Maps feature value to latent space vector.
 - 1 input neuron per latent coordinate.
 - **Pros:** Handles high-cardinality with small vectors; similar concepts near each other.
 - **Cons:** Computationally expensive; needs training data.

Example of One-Hot Encoding

<i>ID</i>	<i>Encryption Algorithm (EA)</i>	<i>EA_AES</i>	<i>EA_RSA</i>	<i>EA_DES</i>
1	AES	1	0	0
2	RSA	0	1	0
3	DES	0	0	1
4	AES	1	0	0

Example of Embeddings

ID	Word	Word Latent Space
1	King	[0.5, 2, 1]
2	Queen	[1.7, 1.8, 0.5]
3	Man	[0.7, 2, 0.2]
4	Woman	[1.9, 1.8, -0.3]

Data Transformation: Numerical Attributes

- 1 input neuron per feature.
- **Normalization (min-max scaling):**
 - Scales using feature min/max.
 - Formula:

Mathematical Expression for Normalization

$$z = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- x = feature, $x_{\max}$ = max, $x_{\min}$ = min.
- **Pros:** Fixed range.
- **Cons:** Outlier-sensitive.
- Example: For temperatures from 0°C to 100°C, a value of 25°C becomes $z = \frac{25-0}{100-0} = 0.25$.
- **Standardization (Z-score scaling):**
 - Scales using mean/variance.
 - Formula:

Mathematical Expression for Standardization

$$z = \frac{x - \mu}{\sigma}$$

- x = feature, μ = mean, σ = variance.

- **Pros:** Handles outliers.
- **Cons:** No boundaries.
- Example: For heights with $\mu = 170$ cm and $\sigma = 10$ cm, a height of 180 cm gives $z = \frac{180-170}{10} = 1$.
- Tools: Use `sklearn` for transformations.

Code Example for Standardization

```
from sklearn.preprocessing import StandardScaler
import numpy as np

data = np.array([[1, 2], [3, 4], [5, 6]])
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
print(scaled_data)
```

Splitting Data

Correct splitting ensures performance estimation and generalization. This step divides the dataset to train, tune, and evaluate the model without bias, promoting fair assessment.

- Split into:
 - **Train:** Train model.
 - **Validation:** hyperparameters via metrics.
 - **Testing:** Assess generalization (similar to validation on unseen data).
- Validation techniques available.
- Address:
 - Class distribution.
 - Misclassification cost.
 - Dataset size.
 - Data leakage.
- Tools: Use `sklearn` for splitting.

Splitting Data: Validation Techniques

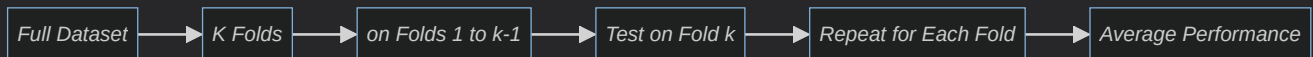
Use stratified sampling for partitions without replacement. This maintains class proportions across splits.

- **Bootstrap:**
 - Sampling with replacement.
- **Hold-out:**
 - Fixed partitioning (e.g., 70% train, 20% validation, 10% test).
 - For large datasets; repeatable.
- **Cross validation:**
 - k disjoint folds.
 - **k-fold:** Train on $k - 1$, test on 1; repeat all folds.
 - Reliable estimation; not for very large/small datasets.
- **Leave-one-out:**
 - $k = n$ cross-validation; for very small datasets.

Example of Scikit-Learn Cross-Validation

Dataset folds, e.g., 40/60/20/30/20/30 for train/validation/test.

The diagram below shows a k-fold cross-validation process:



Splitting Data Problems: Class Distribution

Unbalanced classes (e.g., 99% majority) bias training toward majority. To mitigate, balance the data or adjust the learning process.

- **Data-Level (Resampling):**
 - **Oversampling Minority:**
 - Duplicate/synthetic samples.
 - **SMOTE:** Interpolates minority samples.
 - **ADASYN:** More synthetics for hard instances.
 - **Under-sampling Majority:**
 - Remove majority samples.
 - **Cluster-based:** Select representatives via clustering.
- **Algorithm-Level:**
 - **Class Weights:** Higher loss for minority.

- **Focal Loss:** Focuses on hard samples with weights/factor.
- **Metrics:**
 - Avoid misleading (e.g., 99% accuracy by always predicting majority).
 - Use per-class or weighted (micro avg).

Example of Handling Imbalance

Imbalanced → Oversampled → Undersampled. Weights: 0.2 / 0.8.

Splitting Data Problems: Cost of Misclassification

Some class errors cost more (e.g., missing malware vs. false positive). Prioritize by adjusting the model to penalize costly errors.

- Use metrics focusing on relevant classes.
- **Class Weights:** Higher for minority.
- **Focal Loss:** Focuses on hard samples.

Important Caution on Misclassification Costs

In cybersecurity, missing a malware instance could be catastrophic, so prioritize recall for minority classes.

Splitting Data Problems: Dataset Size

Small datasets unfit for hold-out. Opt for techniques that maximize data usage.

- Use cross-validation/leave-one-out.
- **Data augmentation:**
 - **SMOTE:** Interpolates minority.
 - **ADASYN:** More for hard instances.
- Learn representations first:
 - Fine-tune pre-trained models.
 - Semi-supervised with unlabeled data.

Splitting Data Problems: Data Leakage

Training data influences validation. This contamination leads to overly optimistic performance estimates.

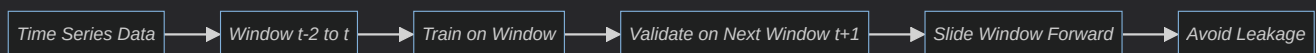
- **Temporal correlation example:**
 - Train: $t - 1$ (0), $t + 1$ (10); validate t (5) – easy, but real prediction lacks $t + 1$.

- **Biasing features:**
 - E.g., DDoS only on port 80; model learns port association, harming generality.
- Solutions:
 - Time-based splitting for temporal patterns.
 - Avoid identifiers/pseudo-identifiers/bias.
 - **Image chunk example:** Train c_0, c_2 ; validate c_1 – too easy.

Examples of Avoiding Leakage

Dataset → Sliding/expanding window. c_0, c_2, c_1 .

The following diagram depicts a sliding window for temporal splitting to avoid leakage:



Deep Learning Architectures

Architectures address specific tasks. Each type is designed for particular data structures, from tabular to sequential or spatial.

- **Feed-Forward Neural Networks (FFNN):**
 - For numerical data (e.g., malware classification on features).
 - Standalone or post-other NNs (e.g., after image encoding).
- **Recurrent Neural Network (RNN):**
 - For sequential data.
 - Chains FFNNs remembering history (e.g., traffic volume prediction, next word).
- **Graph Neural Network (GNN):**
 - For graphs.
 - Aggregates/transforms neighborhood info (e.g., malware detection, social/fraud analysis).
- **Convolutional Neural Network (CNN):**
 - For images.
 - Extracts features via convolutional filters; used in cybersecurity (e.g., malware as image + FFNN).

Deep Learning Architectures

Specialized neural network designs tailored to data types, such as FFNN for tabular data and RNN for sequences, enabling task-specific feature extraction and prediction.

Feed-Forward Neural Network (FFNN)

Starts from basics. FFNNs process inputs through layers without loops, making them suitable for non-sequential data.

Preprocessing and Transforming Data for FFNN

- Encode non-numerical to numbers (e.g., categorical, labels, dates, time).
 - **Note:** Numerical features (e.g., port) may be categorical.
- Normalization/standardization improves performance. These steps ensure features are on similar scales, preventing dominance by larger-range variables.

Link to Prerequisites

See [Linear Algebra](#) for understanding scaling and [Neural Networks](#) for foundational concepts.

FFNN Architecture

1. **Input layer:** Nodes based on preprocessing/transformation.
2. **Hidden-layers:** Few initially; increase if needed.
 - Nodes: Powers of 2 for efficiency.
 - Activation: `ReLU`, `Leaky ReLU`, `Sigmoid`, `tanh` based on depth.
3. **Output layer:** Nodes by classes.
 - Activation: `Sigmoid`, `SoftMax` per problem/loss.
4. **Weights initialization:** Automatic (set seeds for reproducibility); manual for experiments.

Diagram: Data → Hidden.

The diagram below visualizes a basic FFNN architecture:



Optimizing the FFNN Architecture

5. **Input data:** All at once or mini-batches?

6. **Loss Function:** Weights for unbalanced?

7. **Optimizer:** Epochs, learning rate?

- Overfitting: Modify weights, class weights, add dropout.
- Strong overfitting: Restart from preprocessing/architecture.

Diagram: Data → Hidden.

Summary of Hyperparameters and Best Practices for FFNN

Hyperparameter	Best Practices
# Layers	Start with 3-5 hidden layers, increase if needed.
# Neurons per Layer	Use powers of 2 (64, 128, 256), avoid too many.
Activation	<code>ReLU</code> , <code>LeakyReLU</code> , etc. for hidden layers, <code>Softmax</code> / <code>Sigmoid</code> for output.
Weight Initialization	PyTorch automatically initializes the weights with different methods based on the activation function in the layer. Manual initialization is possible.
Batch Size	32-256, tune experimentally.
Loss Function	Choose based on the task.
Optimizer	<code>Adam</code> (Default), <code>SGD</code> + Momentum for generalization.
Learning Rate	Start with 0.001, use LR scheduler.
Epochs & Early Stopping	Monitor validation loss, stop if overfitting. The number of epochs can drastically change based on the optimizer and Learning Rate!
Regularization	Dropout (0.2-0.5) + L2 (0.01).

Loading the Data: From Raw Data to Tensor

Load prepared data as **Tensors** (multi-dimensional matrices of single-type numbers; 1D possible). Tensors are the fundamental data structure in PyTorch, enabling efficient computation on CPUs or GPUs.

- Specify dtype if mismatch (e.g., labels `torch.Long` not `torch.float32`).

Tensors in PyTorch

Multi-dimensional arrays that serve as the core data structure for efficient numerical computations in PyTorch.

Method	Use Case
<code>torch.tensor()</code>	General from lists, tuples, matrices.
<code>torch.zeros()</code> , <code>torch.ones()</code> , <code>torch.arange()</code>	Specific tensors; zeros for masks via multiplication.
<code>torch.rand()</code> , <code>torch.randn()</code> , <code>torch.randint()</code>	Random values.

Example of Creating a Tensor

```
import torch

# From a list
data = torch.tensor([1.0, 2.0, 3.0])
print(data)
```

Loading the Data: Using Mini Batches

Combine tensors via `TensorDataset()` for features/labels. Mini-batches allow efficient training by processing subsets of data, reducing memory usage and speeding up gradient updates.

1. `TensorDataset()` without mini-batches.
 - **Warning:** Memory issues.
2. With `DataLoader()` for mini-batches: Specify dataset, batch size, shuffle.
 - **Shuffle:** Random order per epoch.
 - **Pro:** Improves generalization, reduces overfitting.
 - **Warning:** Preserve order in sequential data; shuffle sequences, not within.
- Validation/test: No shuffle (prediction unchanged; aids loss comparison).

Hyperparameter	# Layers	# Neurons per Layer	Activation	Weight Initialization	Batch Size	Loss Function	O
----------------	-------------	------------------------------	------------	--------------------------	---------------	------------------	---

Create the First Neural Network Architecture: With 1 Linear Layer

Define via class. This basic setup demonstrates a simple linear transformation without non-linearities.

- Class for architecture.
- `__init__`: Parameters (e.g., input neurons, layers, nodes, activation).
 - `super`: Inherits NN.
 - Hidden: 32 neurons, input size.
 - Output: Previous size; nodes by task/classes/loss.
 - Last output: **logit** (raw, unnormalized; $[-\infty, \infty]$).
- `forward`: Data flow.
 - No non-linearity (linear only).

Hyperparameter	# Layers	# Neurons per Layer	Activation	Weight Initialization	Batch Size	Loss Function	O
----------------	-------------	------------------------------	------------	--------------------------	---------------	------------------	---

Example Architecture Details (PyTorch Class Definition)

```
import torch.nn as nn

class SimpleFFNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(SimpleFFNN, self).__init__()
        self.linear1 = nn.Linear(input_size, hidden_size)
        self.linear2 = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        x = self.linear1(x)
```

```
x = self.linear2(x)
return x # Logits
```

Create the First Neural Network Architecture: Add Nonlinearity

- Add activation in `forward`.
- Different per layer.
- Output: Apply for probabilities (e.g., `softmax` for multiclass).
 - Often omitted (loss handles).

Hyperparameter	# Layers	# Neurons per Layer	Activation	Weight Initialization	Batch Size	Loss Function	O
----------------	-------------	------------------------------	------------	--------------------------	---------------	------------------	---

Example (Adding Activation)

```
import torch.nn as nn
import torch.nn.functional as F

class SimpleFFNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(SimpleFFNN, self).__init__()
        self.linear1 = nn.Linear(input_size, hidden_size)
        self.linear2 = nn.Linear(hidden_size, output_size)

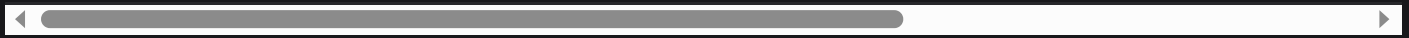
    def forward(self, x):
        x = F.relu(self.linear1(x)) # Nonlinearity in hidden layer
        x = self.linear2(x) # Logits for output
        return x
```

Specify How to Train the Network: Loss & Optimizer

- `criterion`: Loss (e.g., `CrossEntropyLoss()` for multiclass; expects logits, no softmax).
- Check docs; weights for balance.

- **Optimizer:** On parameters; lr, L2, momentum. These components define how the model learns, with loss measuring error and optimizer updating.

Hyperparameter	# Layers	# Neurons per Layer	Activation	Weight Initialization	Batch Size	Loss Function	O
----------------	-------------	------------------------------	------------	--------------------------	---------------	------------------	---



Example (Loss and Optimizer)

```
import torch.optim as optim
import torch.nn as nn

# Assuming model is defined
criterion = nn.CrossEntropyLoss() # For multiclass, expects logits
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=0
```



Train and Validate the Network: The Training Loop (No Mini-Batches)

- `num_epochs` ; track losses for convergence/overfitting. This loop iteratively improves the model by computing gradients and updating parameters.
 1. Train mode.
 2. Zero gradients.
 3. Forward.
 4. Loss.
 5. Backward.
 6. Step.
 7. Append train loss.
- Validation (if available):
 1. Eval mode.
 2. No gradients.
 3. Forward.
 4. Loss; append.
- Print losses; plot trends- Loss not converged.

Example (Training Loop Without Mini-Batches)

```
num_epochs = 100
train_losses = []
val_losses = []

for epoch in range(num_epochs):
    model.train() # Training mode
    optimizer.zero_grad() # Reset gradients
    outputs = model(train_data)
    loss = criterion(outputs, train_labels)
    loss.backward()
    optimizer.step()
    train_losses.append(loss.item())

    if val_data:
        model.eval() # Validation mode
        with torch.no_grad():
            val_outputs = model(val_data)
            val_loss = criterion(val_outputs, val_labels)
            val_losses.append(val_loss.item())

    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}
```

Train and Validate the Network: The Training Loop (With Mini-Batches)

1. `DataLoader` for batches/shuffle.
2. Loop over batches.
3. Epoch loss: Weighted average by batch size/dataset size. This approach is more scalable for large datasets.

Hyperparameter	# Layers	# Neurons per Layer	Activation	Weight Initialization	Batch Size	Loss Function	O
----------------	-------------	------------------------------	------------	--------------------------	---------------	------------------	---

Example (Training Loop With Mini-Batches)

```

from torch.utils.data import DataLoader

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

for epoch in range(num_epochs):
    model.train()
    epoch_loss = 0.0
    num_batches = 0

    for batch_data, batch_labels in train_loader:
        optimizer.zero_grad()
        outputs = model(batch_data)
        loss = criterion(outputs, batch_labels)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item() * batch_data.size(0)
        num_batches += 1

    avg_train_loss = epoch_loss / len(train_dataset)
    train_losses.append(avg_train_loss)

    # Similar for validation without gradients

```

Improve Overfitting with Regularization

1. Dropout layers:

- Rate 0.2-0.5; zeros nodes in train (used in eval).
- After 2nd/4th hidden.

2. **L2:** `weight_decay` in Adam/AdamW. These techniques prevent the model from memorizing training data, enhancing generalization.

Hyperparameter	# Layers	# Neurons per Layer	Activation	Weight Initialization	Batch Size	Loss Function	O
----------------	-------------	------------------------------	------------	--------------------------	---------------	------------------	---

Example (With Dropout and L2)

```
class FFNNWithRegularization(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, dropout_rate):
        super().__init__()
        self.linear1 = nn.Linear(input_size, hidden_size)
        self.dropout1 = nn.Dropout(dropout_rate)
        self.linear2 = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        x = F.relu(self.linear1(x))
        x = self.dropout1(x)
        x = self.linear2(x)
        return x

# Optimizer with L2
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=0.001)
```

Weights Initialization

- Custom for experiments (e.g., small weights cause vanishing gradients).
- PyTorch default vs. manual. Proper initialization ensures stable training by setting initial weights to appropriate scales.

Example (Auto vs Manual Initialization)

- *Auto: PyTorch default.*
- *Manual: Very small.*

Hyperparameter

Layers

Neurons
per
Layer

Activation

Weight
Initialization

Batch
Size

Loss
Function

O

Example (Manual Initialization)

```
def init_weights(m):  
    if isinstance(m, nn.Linear):  
        torch.nn.init.xavier_uniform_(m.weight) # Or other method  
        m.bias.data.fill_(0.01)  
  
model.apply(init_weights)
```

Miscellaneous for FFNN

- **Device:** GPU for deep learning (`cuda` if available); `to(device)` for model/data.
 - Load to GPU before, dump after to avoid memory issues.
 - Small data: CPU fine.
- **Reproducibility:** Set seeds for random (initialization, numbers, splits).
 - Hardware may vary results.

Example (Device and Seed)

```
import torch  
  
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')  
model.to(device)  
data = data.to(device)  
  
torch.manual_seed(42) # For reproducibility
```

Demo for FFNN

- Interactive: <https://playground.tensorflow.org/>
- Try PyTorch!

Recurrent Neural Network (RNN)

Theory: Why Do We Need RNN?

Standard NNs handle fixed vectors independently of order; cannot predict sequences (e.g., next position). RNNs address this limitation by incorporating temporal dependencies.

RNNs model sequences using history. This allows the network to "remember" previous inputs, enabling predictions based on context.

Example of Sequence Prediction

Current t ; possible next. Previous $t - 1, t - 2, t - 3, \dots$

Recurrent Neural Networks

Handle variable-length sequences $x(t)$ via evolving **state**. The state captures information from prior timesteps, making RNNs ideal for dynamic data.

Applications:

- Translation.
- Time-series.
- Speech.
- POS tagging.

Representational Power of RNNs

FFNNs not **Turing-Complete** (no loops/control). They lack the ability to handle arbitrary computations due to their acyclic nature.

RNNs Turing-Complete: Simulate programs; map any input/output sequences. Their recurrent loops provide the computational power for complex sequence modeling.

FFNN vs RNN in a Nutshell

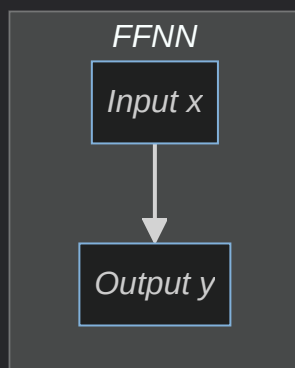
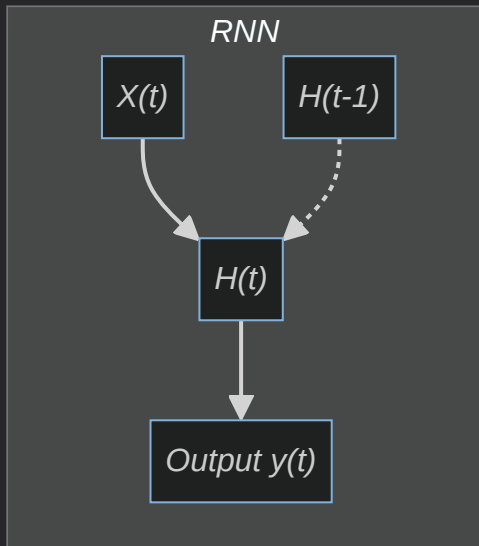
- **FFNN**: Input $x \rightarrow$ output y .
- **RNN**: Input $x(t) + h(t - 1) \rightarrow y(t) + h(t)$; weights w, w' trained. The hidden state $h(t)$ carries memory forward.

Multiple neurons/layers. This comparison highlights how RNNs extend FFNNs with recurrence.

Diagram:

- *FFNN: $h(t-1)$ Input x w Output $y(x)$.*
- *RNN: Input $x(t)$ Output $y(t)$ state retroaction w .*

The diagram below contrasts FFNN and RNN structures:



RNN Architectures: Input/Output Variations (Sequence Modelling Application)

- **One-to-One:** One timestamp; classification/regression.
- **Many-to-One:** Sequence $\rightarrow$ single output (final hidden encodes); e.g., sentiment, cyberattack logs.
- **One-to-Many:** Single $\rightarrow$ sequence; e.g., image captioning, response plans.
- **Many-to-Many:** Sequence-to-sequence (variable lengths); e.g., translation, anomaly detection.

RNN Architectures

1. **Mono-directional RNN:** Sequential forward; hidden from input + previous.
 - **Pros:** Real-time efficient.

- **Cons:** Long dependencies via vanishing/exploding gradients.
- **Training:** BPTT (backprop for sequences).
- Applications: Classification (benign/malicious), anomaly detection.

Backpropagation Through Time (BPTT)

An extension of backpropagation applied to unrolled RNNs across time steps to compute gradients for sequence data.

2. Bi-directional RNN (Bi-RNN): Forward/backward; two stacked RNNs (full trajectory known).

- **Pros:** Past/future context (e.g., speech).
- **Cons:** Costly; not real-time.
- **Training:** Separate BPTT; combine outputs.
- Applications: Phishing emails, insider threats via logs.

3. Long Short-Term Memory (LSTM): Gates (input, forget, output) solve vanishing gradients.

- **Pros:** Long dependencies.
- **Cons:** Expensive vs. vanilla.
- **Training:** BPTT; gates mitigate issues.
- Applications: Long sequences.

LSTM Cell Diagram

See [Neural Networks](#) for more on gated architectures.

RNN Architectures: Internal Structure

RNN maintains h_t for prior info. This hidden state evolves over time, accumulating sequence information.

At t :

Mathematical Expression for Hidden State Update

$$h_t = F_1(W_{hh}h_{t-1} + W_{xh}x_t + b_{hh})$$

- h_{t-1} previous hidden; x_t input; W_{xh} , W_{hh} weights; b_{hh} bias; F_1 activation (`Tanh` , not `ReLU` for gradients).
- $y_t = h_t$.

Diagram: $h_{t-1} \ w_{hh} \ w_{xh} \ b_{hh} \ F_1 \ h_t \ y_t$ RNN cell.

RNN Architectures: Prediction

y_t size = hidden/cell states. Outputs can be generated at each step or aggregated.

Final prediction: Last y_t/h_t + linear layer (optional dropout/normalization/activation).

Mathematical Expression for Output Prediction

$$y_t = F_2(W_{yh}h + b_{yh})$$

- h_t hidden; W_{yh}, b_{yh} output weights/bias; F_2 activation if needed.

Diagram: $h_{t-1} \ x_t \ w_{hh} \ w_{xh} \ b_{hh} \ F_1 \ h_t \ F_2 \ b_{yh} \ y_t \ \sum W_{yy}y_t$.

Feed-Forward Neural Networks Revisited

Single-layer FFNN, multiple outputs; no sequence. FFNNs treat each input independently, ignoring temporal order.

Input x_t at specific t .

Handling Individual Time Steps

Apply same NN per step; independent vectors; separate errors. Without recurrence, steps are isolated.

No sequence: x_0, x_1 not influence $\hat{y}_2$.

Chain via links: h_t state propagated; $\hat{y}_t = f(x_t, h_{t-1})$ (input + memory). Recurrence connects steps through the hidden state.

Unrolled: Right visualization of left recurrent.

Formalization: Recurrent Neural Network (RNNs)

Recurrent relation per step: $h_t = f(W, x_t, h_{t-1})$. This shared function across time enables parameter efficiency.

Same weights/function per step; evaluate sequentially.

Mathematical Expression for RNN Recurrence

$$h_t = fW(x_t, h_{t-1})$$

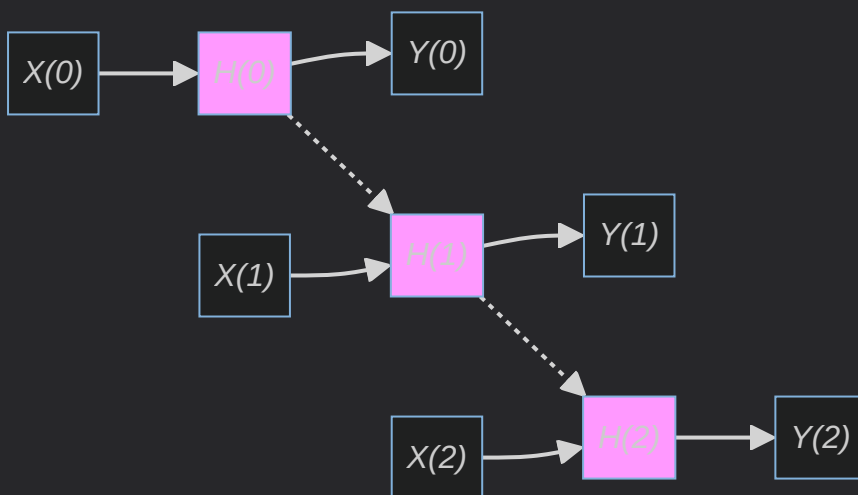
RNNs: Computational Graph Across Time

1. **Compact:** Single node loop for hidden; input varies, output per t . This represents the folded view for efficiency.
2. **Unrolled:** $x_t \rightarrow y_t$; matrices W_{xh} (input $\rightarrow$ hidden), W_{hh} (hidden $\rightarrow$ hidden), W_{hy} (hidden $\rightarrow$ output) shared/updated per step. Unrolling reveals the full temporal dependencies.

Diagram: = $\hat{y}_t \ x_t \dots \dots$ RNN $W_{xh} \ W_{xh} \ W_{xh} \ W_{xh} \ W_{hy} \ W_{hy} \ W_{hy} \ W_{hy} \ W_{hh} \ W_{hh} \ W_{hh}$.

- Reuse weights post-training.

The diagram below shows an unrolled RNN computational graph:



RNNs: Backpropagation Through Time

Backprop + time: Sum losses across sequence for total L . BPTT extends standard backpropagation to recurrent structures.

BPTT: Backpropagate errors from end to start across steps (Werbos, 1990).

Standard RNN Gradient Flow

Gradient wrt h_0 : Many W_{hh} factors. Repeated multiplications can amplify or diminish gradients over long sequences.

>1: **Exploding** – clip gradients. This causes unstable training; gradient clipping bounds updates.

<1: **Vanishing** – from activation, init, architecture; use `Tanh` not `ReLU`. Vanishing gradients hinder learning long dependencies.

Important Caution on Gradient Issues

Exploding gradients require clipping, while vanishing ones necessitate better architectures like LSTM.

The Problem of Long-Term Dependencies

Long sequences (e.g., book words) need memory for past → future. Early information must influence distant predictions.

Short memory destroys learning. Gradients fade, making distant connections hard to learn.

Solutions:

1. Proper activation (`ReLU`).
2. Weight init to avoid vanishing.

More Robust Solution: Create a Smarter Neuron - A Gated Cell

Gating: Gates decide remember/forget. This mechanism selectively updates the state, preserving important information.

Architectures: LSTM, GRU; focus LSTM.

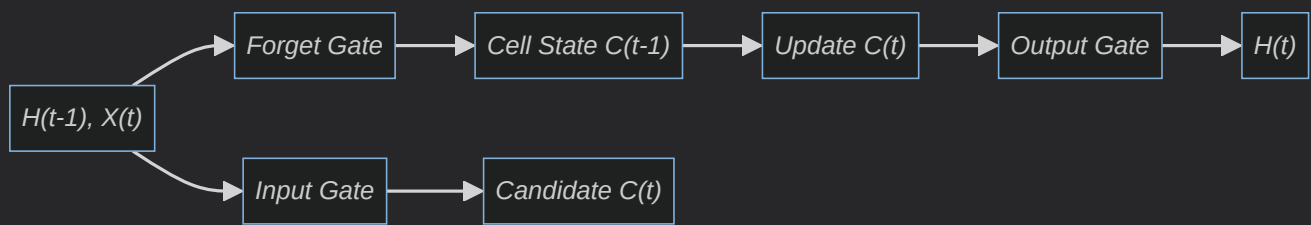
LSTM (Long Short-Term Memory): Key Idea

1997 Hochreiter/Schmidhuber; solves vanishing via gates/cell state. The cell state acts as a "highway" for information flow.

1. Hidden + cell state.
2. Gates: Store/update/filter info.
3. BPTT with uninterrupted gradients.

1) Forget 2) Input 3) Update 4) Output.

The diagram below outlines the LSTM processing flow:



LSTM: Forget Gate

Decides discard/keep past cell state; sigmoid 0-1. This gate filters irrelevant past information.

1. $h_{t-1}, x_t \rightarrow f_t$ (sigmoid).
 2. $f_t \times C_{t-1}$.
- $f_t \approx 1$: Keep; ≈ 0 : Forget.

LSTM: Input Gate

Decides add new to cell; sigmoid filter + `tanh` candidate. This controls what new information to incorporate.

1. $h_{t-1}, x_t \rightarrow i_t$ (sigmoid).
 2. $\tilde{C}_t$ (`tanh`); update $C_t = f_t \times C_{t-1} + i_t \times \tilde{C}_t$.
- $i_t \approx 1$: Store new; ≈ 0 : Ignore.

LSTM: Update Gate

Generates candidate $\tilde{C}_t$ (`tanh` from h_{t-1}, x_t); i_t scales addition to C_t . The candidate provides potential updates.

- $\tilde{C}_t \approx 1$: Add; ≈ -1 : No update.

LSTM: Output Gate

Controls C_t to h_t ; sigmoid o_t on h_{t-1}, x_t . This determines the output based on the cell state.

- $h_t = o_t \times \tanh(C_t)$.
- $y_t = \text{activation}(W_y h_t + b_y)$.
- $o_t \approx 1$: Much influence; ≈ 0 : Little.

How to Implement RNN in PyTorch

Recurrent Neural Network: How to Input Data?

- Preprocess: Numbers as 3D matrix (batch, seq_len, features).
- E.g., `[[[1,2,3],[4,5,6]], [[7,8,9],[10,11,12]]]` (2 series, 2 timesteps, 3 features).

- Mono-dimensional: `[[1],[2],[3]]` as matrix.
- Variable lengths: Pad/cut. Padding ensures uniform batch shapes.

Diagram: Data.

Example Code for Creating a 3D Tensor

```
import torch

# Example 3D tensor: batch=2, seq_len=2, features=3
data = torch.tensor([[[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]],
                     [[7.0, 8.0, 9.0], [10.0, 11.0, 12.0]]])
print(data.shape) # torch.Size([2, 2, 3])
```

Recurrent Neural Network: What About the Model?

1. **Type:** Input-output (many-to-one etc.); length (RNN/LSTM); direction (mono/bi).
2. **Input layer:** Features post-preprocessing.
3. **Hidden-size:** Features in h/c .
4. **Hidden layers:** Default; increase; `tanh`.
5. **Padding:** Ignore in processing.
6. **Pre-output:** Linear if needed.
7. **Output layer:** Classes; activation per problem/loss.

Diagram: Data → RNN → RNN.

Recurrent Neural Network: How to Optimize It?

5. **Input:** Shuffle batches?
 6. **Loss:** Weights for unbalanced?
 7. **Optimizer:** Epochs, lr?
- Overfitting: Weights, class weights, dropout; restart preprocessing/architecture if severe.

Diagram: Data → RNN → RNN.

Summary of Hyperparameters and Best Practices for RNN

Hyperparameter	Best Practices
#Architecture	Mono/bi-directional RNN/LSTM based on sequence order/backward learning and memory length.
# Layers	Start with 1-2 RNN layers, increase if needed.
# Hidden Size	Use powers of 2 (64, 128, 256), avoid too many.
Activation	Typically <code>tanh</code> for hidden. Typically, <code>Softmax</code> / <code>Sigmoid</code> /linear for output.
Weight Initialization	PyTorch automatically initializes the weights with different methods based on the activation function in the layer. Manual initialization is possible also for the hidden/cell states.
Time-series with Different Length	If different time-series have different length we may have to add padding/cutting the time-series.
Batch Size	32-256, tune experimentally.
Batch Shuffling	Consider shuffling if different sequences are not related to each other.
Loss Function	Choose based on the task.
Optimizer	<code>Adam</code> (Default), <code>SGD</code> +Momentum for generalization.
Learning Rate	Start with 0.001, use LR scheduler.
Epochs & Early Stopping	Monitor validation loss, stop if overfitting. The number of epochs can drastically change based on the optimizer and Learning Rate!
Regularization	Dropout (0.2-0.5) (Notice can be inserted as separate layers or directly in the RNN nodes as well) + L2 (0.01).
#Architecture After RNN	After the RNN consider if it is required to add additional linear/non-linear layers or architectures based on task and performance.

Loading the Data: From Raw Data to Tensor

Time-series as 3D. This format aligns with RNN expectations for batched sequences.

- Transform 2D (timesteps x features + label) via `create_sequence` to 3D (series x timesteps x features); remove labels.
 - E.g., 35040 timesteps x 7 features → 35033 series x 7 timesteps x 7 features.
- Variable lengths:
 - Mini-batches: Same length within batch; different across.
 - FFNN: Fixed size only.
 - Handle: Pad shorter/cut longer; custom `TensorDataset` with `collate_fn`.
 1. `__init__`: Data setup.
 2. `__getitem__`: Single element.
 3. `collate_fn`: Pad via `pad_sequence` (prefer padding).
 - `DataLoader`: Use `collate_fn` if needed (per-batch variable).

Create a Recurrent Neural Network Architecture

Class as FFNN. RNN classes extend PyTorch's `nn.Module`, leveraging built-in recurrent modules.

1. Class.
2. `__init__`: Input size, hidden, layers; `super`.
 - RNN: `input_size`, `hidden_size`, `num_layers`.
 - Output: `hidden_size` to `output_size`; optional intermediates.
3. `forward`: Flow.

Create a Recurrent Neural Network Architecture: How to Consider the Padding?

- Handle variable: Last hidden for classification.
- Unsqueeze for 2D → 3D.
- `pack_padded_sequence`: Skip padding in RNN.
- Pad back outputs if needed.
- Linear on last hidden (fixed size, avoids padding bias). This ensures fair treatment of sequences.

Create a Recurrent Neural Network Architecture: Different Architecture

- **Bi-directional:**
 - `bidirectional=True`.
 - Linear: `hidden * 2`.
 - Concat forward/backward hiddens.
- **LSTM:**
 - `nn.LSTM`.

- Forward: Use (hn, cn).

Example (Basic RNN Architecture)

```
import torch.nn as nn

class SimpleRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, num_la
        super(SimpleRNN, self).__init__()
        self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch
        self.linear = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        # x shape: (batch, seq_len, input_size)
        out, hn = self.rnn(x)
        # Use last hidden state
        out = self.linear(hn[-1])
        return out
```

Example (Handling Padding)

```
from torch.nn.utils.rnn import pack_padded_sequence

class PaddedRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)
        self.linear = nn.Linear(hidden_size, output_size)

    def forward(self, x, lengths):
        # x: (batch, seq_len, input_size), lengths: batch sequence
        x = pack_padded_sequence(x, lengths, batch_first=True, enforce
        out, hn = self.rnn(x)
        out, _ = torch.nn.utils.rnn.pad_packed_sequence(out, batch_
        # Use last relevant hidden state or hn
```

```
out = self.linear(hn[-1])
return out
```

Example (Bi-directional RNN)

```
class BiRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.rnn = nn.RNN(input_size, hidden_size, bidirectional=True)
        self.linear = nn.Linear(hidden_size * 2, output_size) # 2x

    def forward(self, x):
        out, (hn, cn) = self.rnn(x)
        # Concat forward and backward hidden states
        hn = torch.cat((hn[-2,:,:], hn[-1,:,:]), dim=1)
        out = self.linear(hn)
        return out
```

Example (LSTM)

```
class SimpleLSTM(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, num_la
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, ba
        self.linear = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        out, (hn, cn) = self.lstm(x) # Returns hidden and cell sta
        out = self.linear(hn[-1])
        return out
```

Example (Custom Dataset with Padding)

```
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence

class PaddedDataset(Dataset):
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]

def collate_fn(batch):
    data, labels = zip(*batch)
    lengths = [len(seq) for seq in data]
    data = pad_sequence(data, batch_first=True, padding_value=0)
    labels = torch.tensor(labels)
    return data, labels, lengths

# Usage
dataset = PaddedDataset(sequences, labels)
dataloader = DataLoader(dataset, batch_size=32, collate_fn=collate_
```

Demo for RNN

- Try PyTorch!

References

- [Machine Learning](#)
- [Neural Networks](#)
- [Linear Algebra](#)